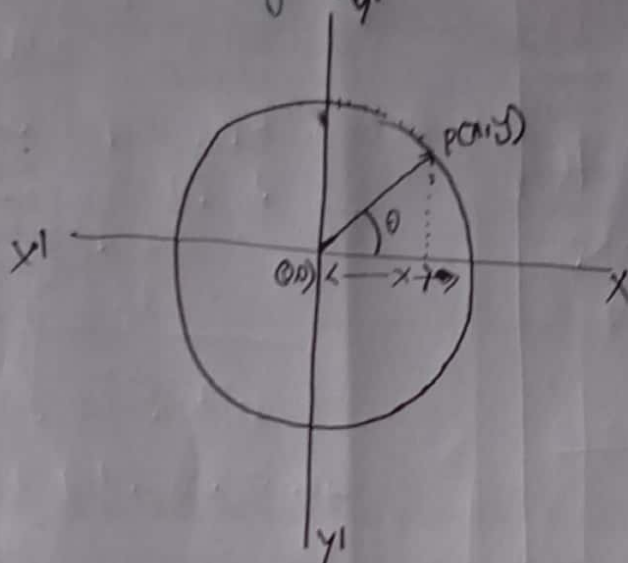


Q Define circle using polynomial method. Write down the

Brenenham's circle Algo $\rightarrow$



We know,
 $\frac{y}{x} = \tan \theta$
 $\frac{y}{x} = \tan 45^\circ$
 $\Rightarrow \frac{y}{x} = 1$
 $\therefore x = y$

$\Rightarrow$ To Define a circle we use,
 second order polynomial eqn.

$$y^2 = r^2 - x^2$$

$\Rightarrow$ With this method,
 each x coordinate in the 90° to 45°
 sector is found by stepping,
 x from 0 to $\frac{r}{\sqrt{2}}$

and,
 each y is found by evaluating
 $\sqrt{r^2 - x^2}$ for each step of x.

Algo:

step 1: set the initial variables

r = circle radius

(h, k) = co-ordinates of the
 center of the circle

x = 0

d = step size

x_end = $\frac{r}{\sqrt{2}}$

step 2: if $x > x_{end}$

stop

step 3: compute $y = \sqrt{r^2 - x^2}$

step 4: using 8 way symmetry
 plotting points,

plot(x+h, y+k) plot(-x+h, -y+k)

plot(y+h, x+k) plot(-y+h, -x+k)

plot(-y+h, x+k) plot(y+h, -x+k)

plot(-x+h, y+k) plot(x+h, -y+k)

step 5: $x = x + d$, step 6: goto(2)

Midpoint Circle Algo:

Step 1: Start Algo

Step 2: Declare P, Q, X, Y, n, d variables

P, Q - coordinates of the center.

R - radius

Step 3: Enter value of R

Step 4: calculate $d = R^2 - 2R$

Step 5: Initialize $x=0, y=R$

Step 6: If $x \geq y$, stop

Step 7: plot 8 points using concept of 8 way symmetry.

putpixel($x + P, y + Q$)
" ($y + P, x + Q$)
" ($-y + P, x + Q$)
" ($-x + P, y + Q$)
" ($-x + P, -y + Q$)
" ($-y + P, -x + Q$)
" ($y + P, -x + Q$)
" ($x + P, -y + Q$)
" ~~0~~

Step 8: Find location of next pixel,

If $d < 0$

$$d = d + 4n + 6$$

$$x = n + 1$$

$$y = y$$

If $d \geq 0$

$$d = d + 4(n - y) + 10$$

$$x = x + 1$$

$$y = y - 1$$

Step 9: go to step 6

Step 10: end algo

Midpoint Circle.

$$x=0, y=R$$

$$P = R - R$$

case 1:

$$P_k < 0$$

$$P_{k+1} = P_k + 2n + 1$$

$$x_n = x + 1$$

$$y_n = y$$

case 2:

$$P_k > 0$$

$$P_{k+1} = P_k + n + 1 - 2y$$

$$x_n = x + 1$$

$$y_n = y - 1$$

Midpoint Circle

Step 2: Repeat while $x \leq y$

plot (x, y)

if ($P < 0$)

then set $P = P + 2n + 1$

else,

$$P = P + 2(n - y) + 1$$

$y = y - 1$ (end if)

$x = n + 1$ (end loop)

Step 3: end

Polynomial Method (ellipse)

- ellipse is 4 way symmetry
- $a_1 \rightarrow$ major axis of ellipse
- $b_1 \rightarrow$ minor axis
- $(h, k) \rightarrow$ center

$\rightarrow$ value of x will be incremented from, 1 to a_1

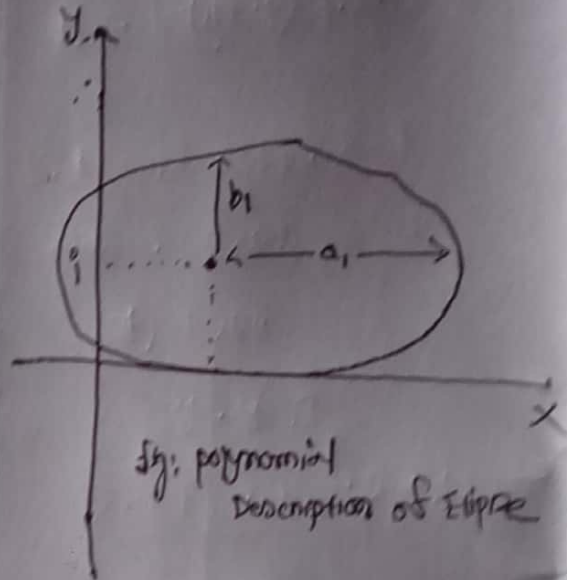
$$y = b_1 \sqrt{1 - \frac{(x-h)^2}{a_1^2}} + k$$

Algo:

1. set initial variables
 a, b - major and minor axis length
 (h, k) - co-ordinates of center
 $x = 0$
 $i = \text{step}$
 $x_{\text{end}} = a$
2. if $x > x_{\text{end}}$ stop
3. compute $y = b \sqrt{1 - \frac{(x-h)^2}{a^2}} + k$
4. plot the 4 points,
plot $(x+h, y+k)$
 $(-x+h, -y+k)$
 $(-y+h, x+k)$
 $(y+h, -x+k)$
5. $x = x + i$
6. go to step 2.

Drawback:

1. requires squaring of values
floating point data required
2. calculations are
- slow
- complex



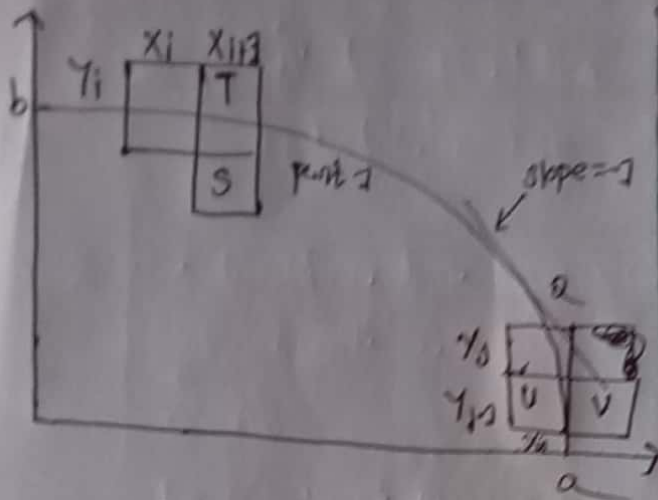
Drawback of Trigonometric method

1. Inefficient method
2. Memory is required to store the value of θ
3. It is not an interactive method.
4. A Table is required to store trigonometric values.

Midpoint Ellipse Algo:

$$f(x,y) = b^2x^2 + a^2y^2 - a^2b^2$$

< 0 (x,y) inside the ellipse
 $= 0$ (x,y) on " "
 > 0 (x,y) outside



Steps in Midpoint Ellipse Algo:

1. Initialize variables a, b

Start with int point (x,y) = (0,b)

2. calculate Decision parameter for the int region (slope between 0 to -1)

$$P = b^2 - a^2b + \frac{a^2}{4}$$

3. Region 1: slope > -1

while $2b^2x \leq 2a^2y$:

If $P_1 < 0$:

$$x = x + 1$$

$$P_1 = P_1 + 2b^2x + b^2$$

If $P_1 \geq 0$:

$$x = x + 1$$

$$y = y - 1$$

$$P_1 = P_1 + 2b^2x - 2a^2y + b^2$$

4. calculate D.p for Region 2 (slope between -1 to 0):

$$P_2 = b^2(x+1)^2 + a^2(y-1)^2 - a^2b^2$$

5. Region 2: slope ≤ -1

while $y \geq 0$:

If $P_2 > 0$:

$$y = y - 1$$

$$P_2 = P_2 - 2a^2y + a^2$$

If $P_2 \leq 0$:

$$x = x + 1$$

$$y = y - 1$$

$$P_2 = P_2 + 2b^2x - 2a^2y + a^2$$

6. plot points.

Two Essential aspects of Transformation:

1. Each transformation can be represented as a single entity by a unique symbol or name.

Ex: 'A' for Translation
'B' for Scaling

2. Two transformations can be combined into a single transformation.

$$C = AB$$

↳ Result of Applying both 'A' and 'B' transformation.

Two complementary points of view for Describing Object Transformation:

1. Geometric Transformation:

↳ Directly applied to each point of the object.

↳ Object is transformed relative to a fixed coordination system or Background

Ex: moving a car while BG is fixed,

2. Co-ordination Transformation:

↳ effect is achieved by transforming the coordination system while object remains stationary.

Ex: keeping the car fixed while moving the BG.

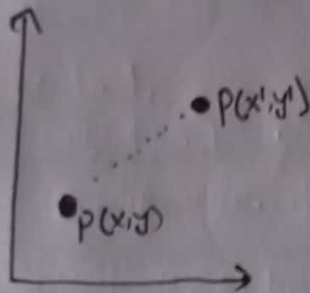
⇒ Types:

1. Translation
2. Rotation
3. Scaling
4. Reflection
5. Shearing.

Types of Transformation:

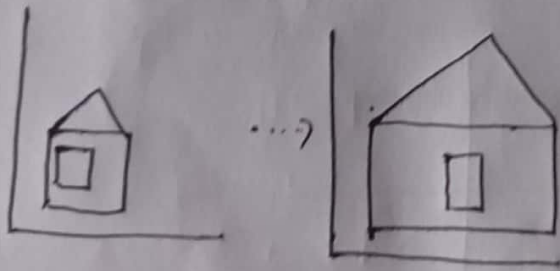
1. Translation:

- moves an object from one position to other without changing
 - shape
 - size
 - orientation



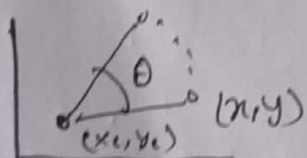
2. Scaling:

- changes the size of the object by multiply the coordinates by a scaling factor.



3. Rotation: changing the angle of the object around pivot point.

- Types
 - Clockwise
 - Anticlockwise



4. Reflection:

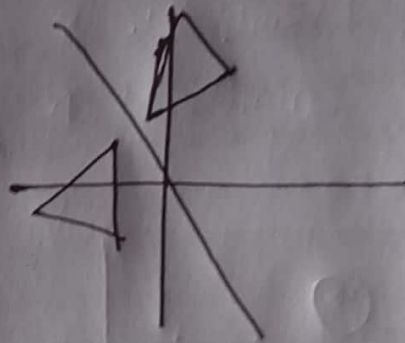
- produces mirror image of an object.

Types:

- Ref. about x -axis
- " y -axis

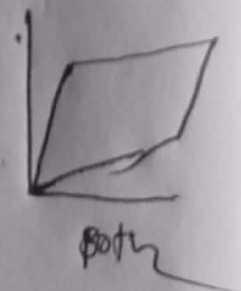
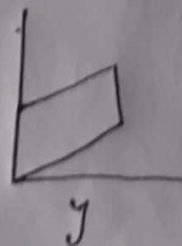
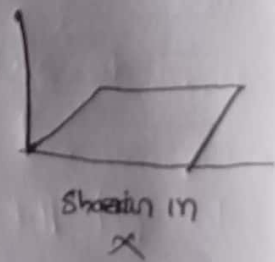
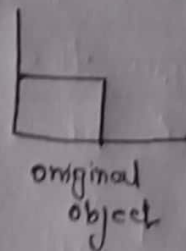
- " perpendicular to xy plane and passing through origin.

- " " line $y=x$



5. Shearing:

- changes the shape of the object.



1. Define clipping. write down the point clipping procedure.

Clipping:

→ It is the process of limiting the rendering of graphics to a specific region.

so that,
only the parts within that region are displayed and anything outside is excluded.

→ can be applied through h/w on SW.

Types:

1. point
2. line
3. Area/polygon
4. curve
5. Text
6. Exterior

Application:

1. Extracts the desired part,
2. Identifies the visible and invisible area in the 3D object,
3. Creating objects with Solid Modeling
4. For Drawing operations
5. Can delete, moving, copying part of an object

point clipping procedure:

→ It is used to check or determine whether a point is inside a window or not.

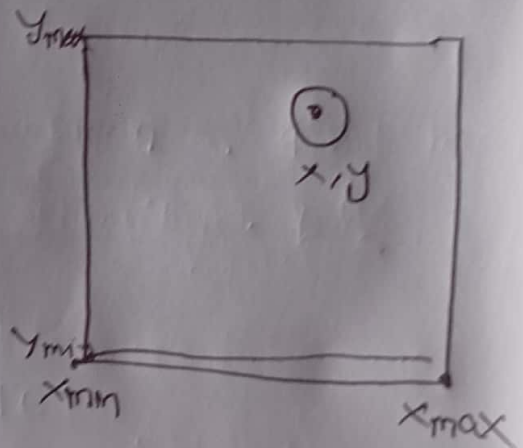
→ For this the following conditions are checked,

1. $X \leq X_{max}$

2. $X \geq X_{min}$

3. $Y \leq Y_{max}$

4. $Y \geq Y_{min}$



→ If any of the above inequalities is false for (X, Y)
↳ point will fall outside the window
↳ will not be considered visible.

Q7 Describe in detail the Cohen-Sutherland algo to clip a straight line segment against a rectangle.

→ It divides the 2D space into regions

→ Efficiently determines whether a line should be

→ Entirely displayed / visible

→ completely discarded / not visible

→ partially clipped →

based on its position relative to a rectangular clipping window.

→ Let $A(x_1, y_1)$ and $B(x_2, y_2)$ are endpoints of the line.

→ coordinates are → x_{min} , y_{min} , y_{max} , x_{max} to draw clipping window

→ Each endpoint of a line is assigned a 4 bit region code
the bits are ordered as Top, Bottom, Right, Left.

1001	1000	1010
0001	0000	0010
0101	0100	0110

→ The centre one will have the code, 0000.

There will be 3 cases for a line.

1. completely inside.

Bitwise OR = 0000

2. completely outside:

Bitwise AND $\neq$ 0000

3. partially visible:

Bitwise AND = 0000

Algorithm exp:

Step 1: Assign region code to endpoints

Step 2: perform Bitwise OR

Step 3: If (Bitwise OR = 0000)

{ completely inside }

else perform Bitwise AND
If (Bitwise AND $\neq$ 0000)
{ completely outside }

else (Bitwise AND = 0000)
{ partially inside/outside }

Step 6: Repeat until step 4 until the line gets completely clipped.

Step 4: case (Top):

$$x_2 = x_1 + (y_{\text{max}} - y_1) / m$$

$$y_2 = y_{\text{max}}$$

case (Bottom)

$$x_2 = x_1 + (y_{\text{min}} - y_1) / m$$

$$y_2 = y_{\text{min}}$$

case (left):

$$x_2 = x_{\text{min}}$$

$$y_2 = y_1 + (x_{\text{min}} - x_1) / m$$

case (right):

$$x_2 = x_{\text{max}}$$

$$y_2 = y_1 + (x_{\text{max}} - x_1) / m$$

Step 5: Update the values of x_1, y_1 based

Midpoint Subdivision Line Clipping

Algo:

Step 1: calculate the position of both endpoints of the line

Step 2: perform OR operation on both these endpoints
~~If AND~~

Step 3: If the OR op gives 0000

then:

Line is guaranteed to be visible.

else:

perform AND operation of both endpoints

If AND $\neq$ 0000

then the line is invisible

else:

AND = 0000

then the line is clipped case.

Step 4: Find midpoint of line

$$X_m = (x_1 + x_2) / 2$$

$$Y_m = (y_1 + y_2) / 2$$

Step 5: check if the midpoints are nearest to the boundary of a window or not.

Step 6: If the line is totally visible or totally rejected repeat step 1 to 5

Step 7: Stop Algo.

Sutherland-Hodgeman polygon clipping:

→ performed by processing the boundary of a polygon against each window corner on edge.

→ There may be 4 possible cases while clipping

→ 1. 1st vertex outside, second vertex inside:

- Add the second vertex

→ Intersection point to output

→ 2. Both vertices inside:

- only add the second vertex to output

→ 3. 1st inside, 2nd outside:

- only add the intersection point

→ 4. Both vertices outside:

- nothing is added to the output.